VNR VJIET Name of the Experiment:

Name of Laboratory: _____

_____ Date: _____

WEEK- 6

Feature Extraction:

1. Principal Component Analysis (PCA)
2. Singular Value Decomposition (SVD)
3. Linear Discriminant Analysis (LDA)
4. Feature Subset Selection

```
# importing all the packages
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import sklearn.preprocessing
import numpy as np
```

```
df = pd.read_csv('iris.csv')
iris = df
df
```

```
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
0      1            5.1           3.5            1.4           0.2
1      2            4.9           3.0            1.4           0.2
2      3            4.7           3.2            1.3           0.2
3      4            4.6           3.1            1.5           0.2
4      5            5.0           3.6            1.4           0.2
..   ...            ...           ...            ...           ...
145  146            6.7           3.0            5.2           2.3
146  147            6.3           2.5            5.0           1.9
147  148            6.5           3.0            5.2           2.0
148  149            6.2           3.4            5.4           2.3
149  150            5.9           3.0            5.1           1.8

            Species
0       Iris-setosa
1       Iris-setosa
2       Iris-setosa
3       Iris-setosa
4       Iris-setosa
..              ...
145  Iris-virginica
146  Iris-virginica
147  Iris-virginica
148  Iris-virginica
```

```
149  Iris-virginica

[150 rows x 6 columns]
```

1. Principal Component Analysis (PCA)

- Principal Component Analysis (PCA) is a data dimensionality reduction method that simplifies complex datasets by identifying and retaining the most important information while discarding less critical aspects. It does this by transforming the data into a new set of orthogonal axes called principal components. The first principal component captures the most significant variation in the data, and subsequent components capture decreasing amounts of variation. This allows for a reduction in the number of variables while maintaining the data's key patterns and trends, making it easier to visualize and analyze high-dimensional data.
- PCA finds applications in various fields, including image analysis, genetics, finance, and machine learning. It is particularly useful for data preprocessing, noise reduction, and feature extraction. However, it assumes linearity in the data and may not perform well when non-linear relationships exist. For such cases, alternative dimensionality reduction techniques like t-SNE or nonlinear autoencoders are more appropriate. In summary, PCA is a valuable tool for simplifying and understanding complex data, enabling more efficient analysis and visualization.

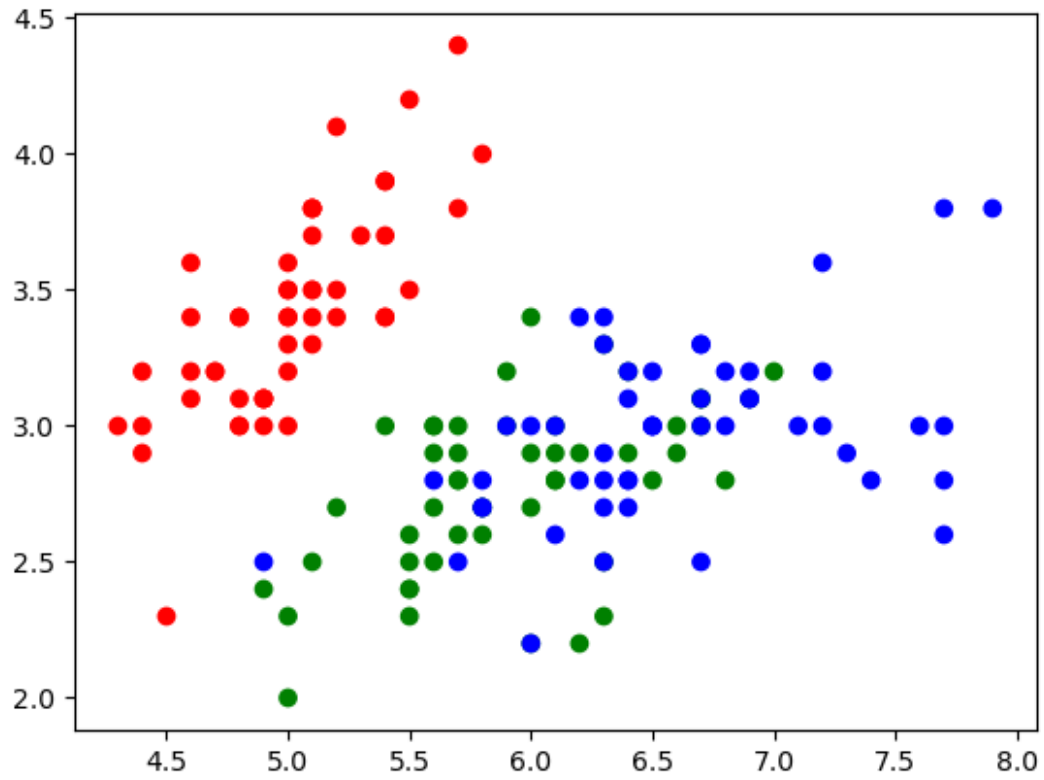
```python
# Determining initial dimensions of the data
x = df[['SepalLengthCm', 'SepalWidthCm','PetalLengthCm', 'PetalWidthCm']]
y = df[['Species']]
print(x.head())
print(y.head())
print(x.shape,y.shape)
```

```
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
0            5.1           3.5            1.4           0.2
1            4.9           3.0            1.4           0.2
2            4.7           3.2            1.3           0.2
3            4.6           3.1            1.5           0.2
4            5.0           3.6            1.4           0.2
       Species
0  Iris-setosa
1  Iris-setosa
2  Iris-setosa
3  Iris-setosa
4  Iris-setosa
(150, 4) (150, 1)
```

```python
# Define a color mapping for the classes
color_mapping = {
    'Iris-setosa': 'red',
    'Iris-versicolor': 'green',
    'Iris-virginica': 'blue'
}

# Map the 'Species' column to colors
colors = y['Species'].map(color_mapping)

# Scatter plot with colors
plt.scatter(x.iloc[:, 0], x.iloc[:, 1], c=colors)
plt.show()
```




```
pca = PCA()
X_new = pca.fit_transform(x)
cov_mat = pca.get_covariance()
cov_mat
```

```
array([[ 0.68569351, -0.03926846,  1.27368233,  0.5169038 ],
       [-0.03926846,  0.18800403, -0.32171275, -0.11798121],
       [ 1.27368233, -0.32171275,  3.11317942,  1.29638747],
       [ 0.5169038 , -0.11798121,  1.29638747,  0.58241432]])
```

```
eig_vals, eig_vec = np.linalg.eig(cov_mat)
print("Eigen Values :\n",eig_vals)
print("Eigen Vectors: \n",eig_vec)
```

```
Eigen Values :
 [4.22484077 0.24224357 0.07852391 0.02368303]
Eigen Vectors:
 [[ 0.36158968 -0.65653988 -0.58099728  0.31725455]
 [-0.08226889 -0.72971237  0.59641809 -0.32409435]
 [ 0.85657211  0.1757674   0.07252408 -0.47971899]
 [ 0.35884393  0.07470647  0.54906091  0.75112056]]
```

```
# PCA for target dimension on the dataset
pca = PCA(n_components=2)
pca.fit(x)
```

```
PCA(n_components=2)
```

```
# Principal components
pca.components_
```

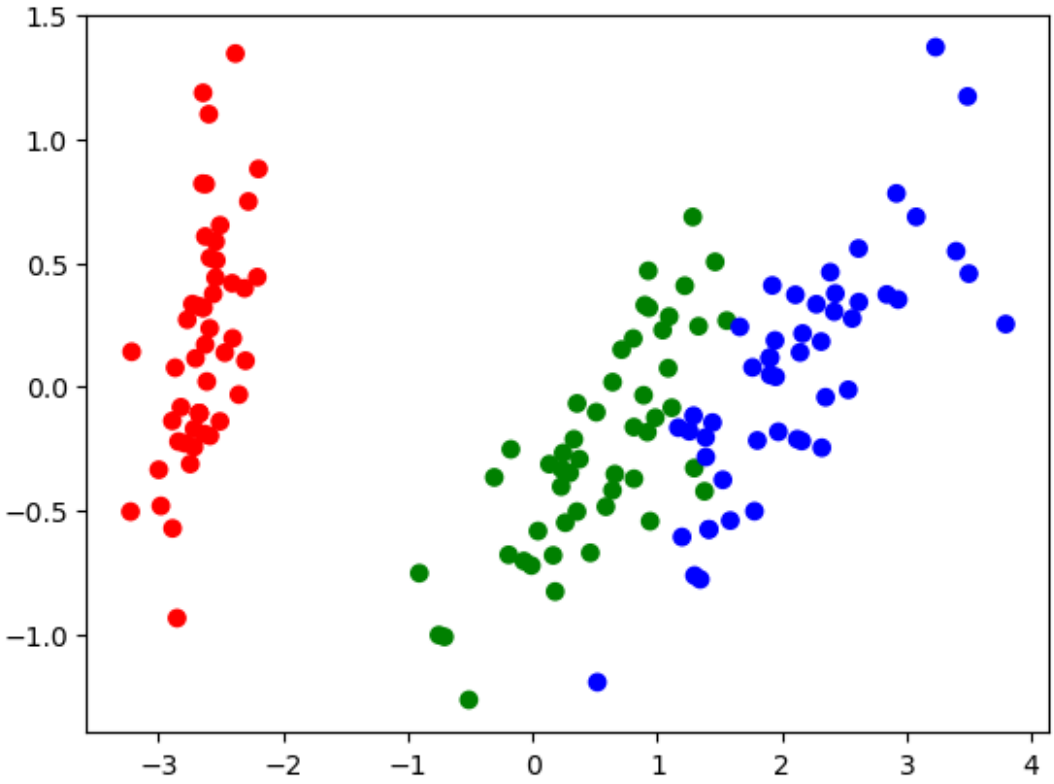
```
array([[ 0.36158968, -0.08226889,  0.85657211,  0.35884393],
       [ 0.65653988,  0.72971237, -0.1757674 , -0.07470647]])
```

```
# Transforming the dataset from 4 dimensions to 2 dimensions using PCA
z = pca.transform(x)
z.shape
```

```
(150, 2)
```

```
# Plotting Scatter Plot:

# Scatter plot with colors
plt.scatter(z[:, 0], z[:, 1], c=colors)
plt.show()
```




```
# Variance ratio of the target dimensions
pca.explained_variance_ratio_
```

```
array([0.92461621, 0.05301557])
```

Observation:

- The three classes are well separated.
- Together, the first two principal components contain 97.76% of the information.
- The first principal component contains 94.46% of the variance and the second principal component contains 5.3% of the variance. The third and fourth principal component contained the rest of the variance of the dataset.

2. Singular Value Decomposition (SVD)

- Singular Value Decomposition (SVD) is a fundamental matrix factorization technique used in linear algebra, data analysis, and machine learning. It decomposes a given matrix into three constituent matrices, which can be thought of as capturing essential information about the original matrix. In SVD, any m x n matrix can be expressed as the product of three matrices: U (m x m), Σ (m x n), and V^T (n x n), where U and V^T are orthogonal matrices, and Σ is a diagonal matrix containing singular values. These singular values in Σ represent the

magnitude of information in the data along each dimension and can be used for dimensionality reduction, noise reduction, and data compression.

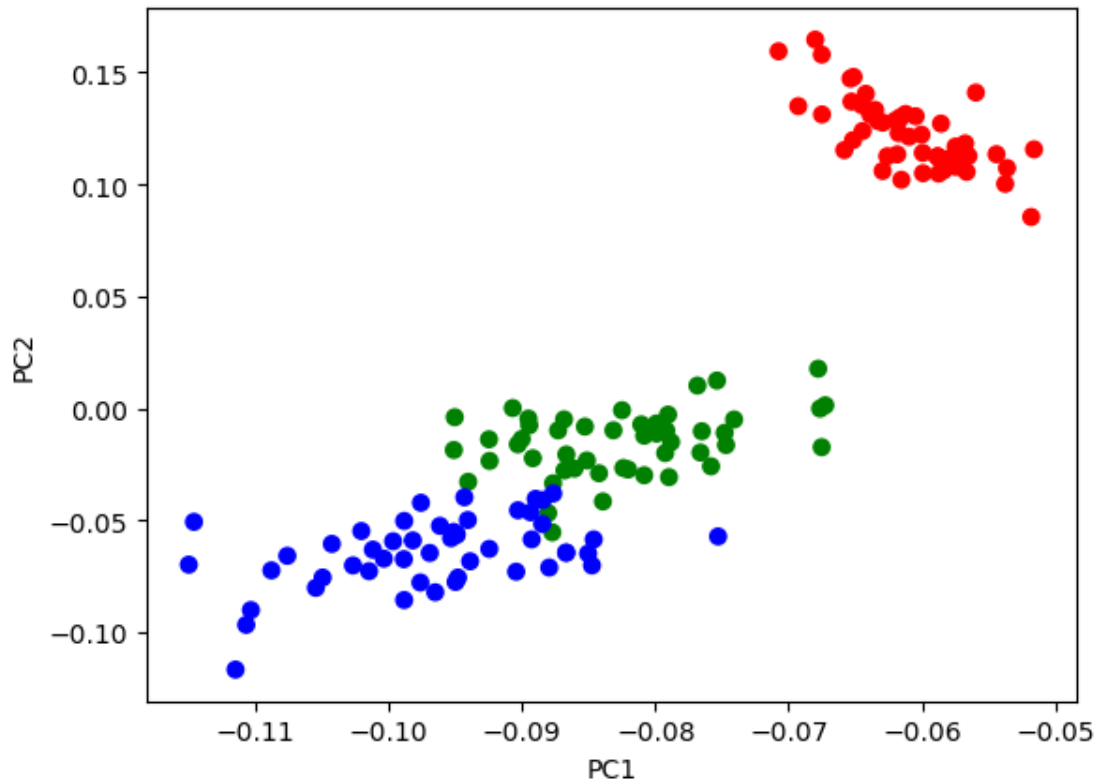
- SVD finds applications in a wide range of fields, including data compression, image processing, natural language processing, and recommendation systems. It is particularly useful in reducing the dimensionality of data, which is beneficial for feature selection, removing noise, and uncovering latent structures in large datasets. Additionally, SVD plays a crucial role in other matrix factorization techniques, such as Principal Component Analysis (PCA) and collaborative filtering in recommendation systems.

```
iris = x.astype(float)
U, S, Vt = np.linalg.svd(iris, full_matrices=False)
print(f"{U.shape}\n{S}\n{Vt}")
```

```
(150, 4)
[95.95066751 17.72295328  3.46929666  1.87891236]
[[-0.75116805 -0.37978837 -0.51315094 -0.16787934]
 [ 0.28583096  0.54488976 -0.70889874 -0.34475845]
 [ 0.49942378 -0.67502499 -0.05471983 -0.54029889]
 [ 0.32345496 -0.32124324 -0.48077482  0.74902286]]
```

- NOTE: numpy.linalg.svd actually returns a Σ that is not a diagonal matrix, but a list of the entries on the diagonal.

```
# After SVD
import matplotlib.pyplot as plt
# Plot the first two columns of U
plt.scatter(U[:, 0], U[:, 1], c=colors)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```



3. Linear Discriminant Analysis (LDA)

- Linear Discriminant Analysis (LDA) is a dimensionality reduction and classification technique commonly used in machine learning and pattern recognition. It aims to find a linear combination of features that maximizes the separation between multiple classes while minimizing the variance within each class. LDA is particularly useful for supervised learning tasks where the goal is to classify data into predefined categories. By transforming the data into a lower-dimensional space, LDA not only reduces dimensionality but also enhances the discriminatory power of the features, making it easier for classifiers to distinguish between different classes. LDA is widely used in applications like face recognition, document classification, and bioinformatics.

- Unlike Principal Component Analysis (PCA), which focuses on capturing overall variance in the data, LDA focuses on maximizing class separability. It achieves this by finding the linear projections of the data that best separate the classes, making it a valuable tool for feature selection and classification tasks where class discrimination is crucial.

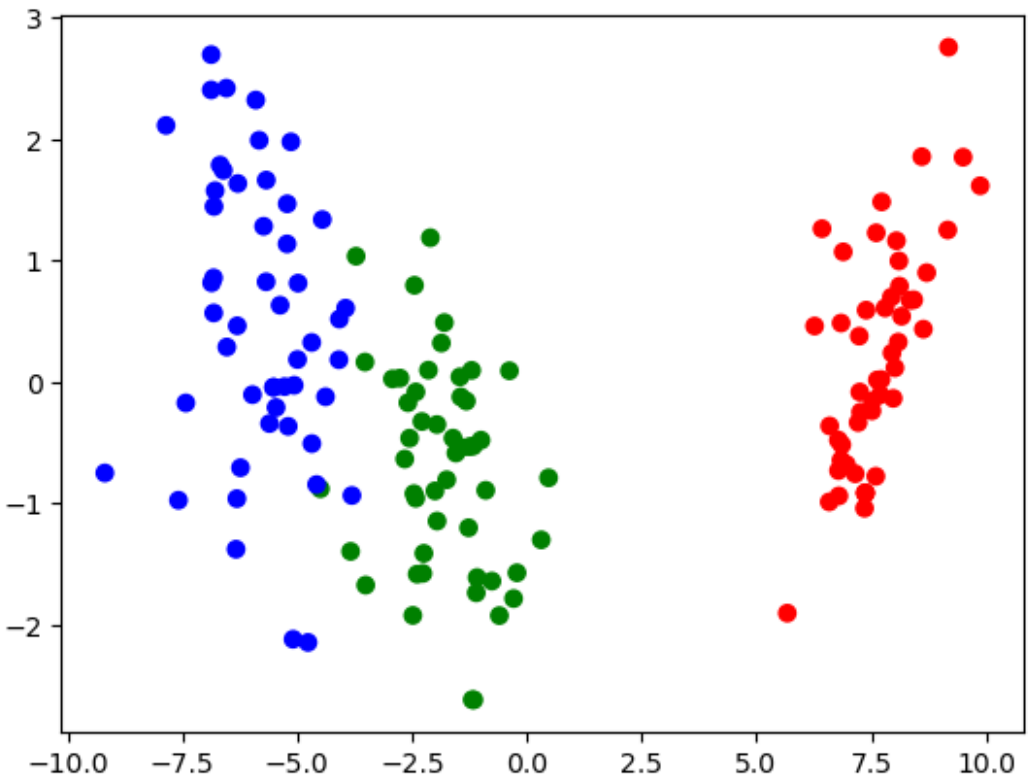
```
lda = LinearDiscriminantAnalysis(n_components=2)
X_r2 = lda.fit(x, y.values.ravel()).transform(x) # 'y' variable should be
reshaped into a 1D array instead of a
# column-vector. ravel() function is used to flatten 'y' into a 1D array

#Getting the variance ratio
lda.explained_variance_ratio_
```

```
array([0.99147248, 0.00852752])
```

```
# visualizing 2D datab in the form of scatter plot
plt.scatter(X_r2[:,0],X_r2[:,1],c=colors)
```

```
<matplotlib.collections.PathCollection at 0x29f88e0bf10>
```



Observation:

- LDA is able to able to separate the classes very well after dimenionality reduction

4. Feature Subset Selection

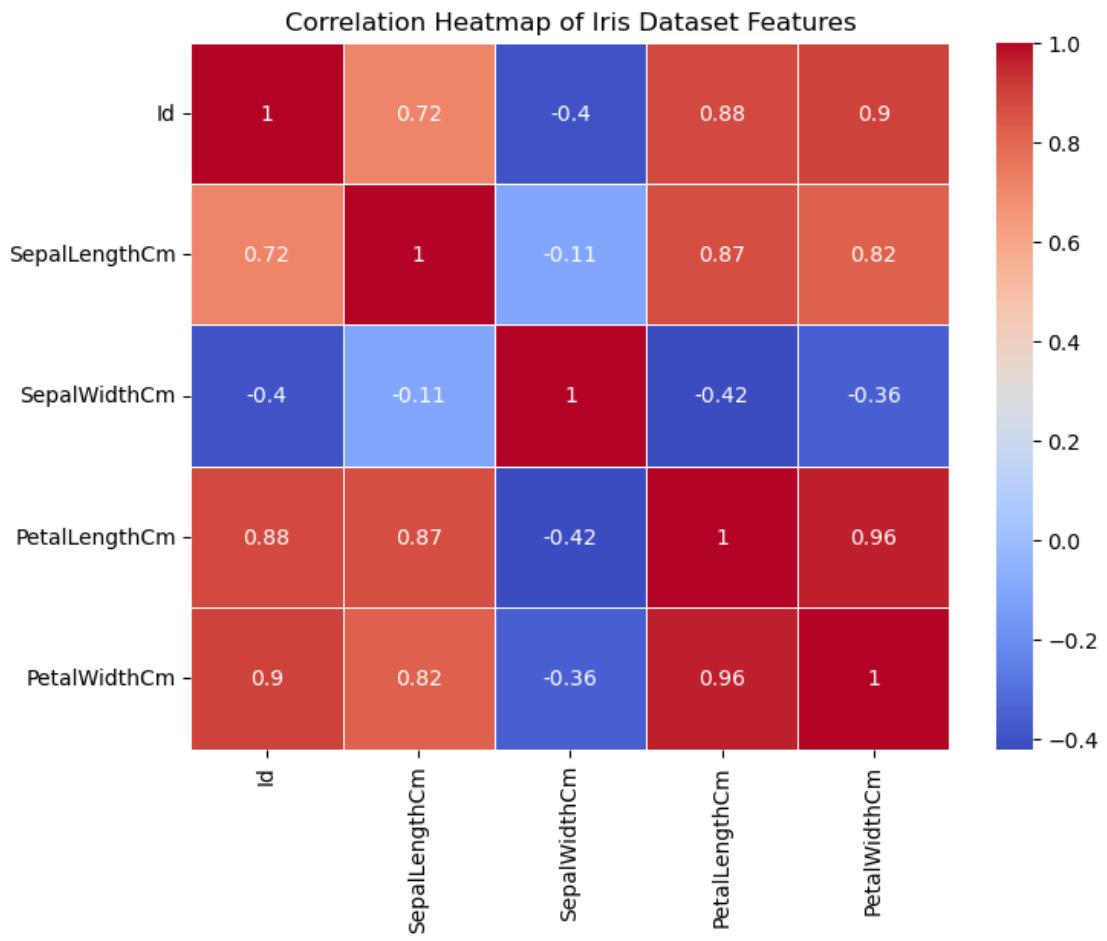
- Feature subset selection is a process of choosing a subset of the most relevant features (variables) from your dataset while eliminating less important or redundant ones. This is done to improve model performance, reduce overfitting, and enhance interpretability. There are various techniques for feature subset selection

4.1: Correlation Matrix with Heatmap

- Correlation states how the features are related to each other or the target variable.
- Correlation can be positive (increase in one value of feature increases the value of the target variable) or negative (increase in one value of feature decreases the value of the target variable)
- The logic behind using correlation for feature selection is that good variables correlate highly with the arget. Furthermore, variables should be correlated with the target but uncorrelated among themselves.
- If two variables are correlated, we can predict one from the other. Therefore, if two features are correlated, the model only needs one, as the second does not add additional information.
- Heatmap makes it easy to identify which features are most related to the target variable, we will plot heatmap of correlated features using the seaborn library.

```
# Create a correlation matrix
correlation_matrix = df.corr()

# Create a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", linewidths=0.
5)
plt.title("Correlation Heatmap of Iris Dataset Features")
plt.show()
```




```
# Load your dataset, assuming it's in a DataFrame named 'df'
# Replace 'your_target_column'('species') with the name of your target var
iable column
# Remove the target column from the DataFrame to focus on feature-feature
correlations
X = df.drop('Species', axis=1)

# Calculate the correlation matrix
correlation_matrix = X.corr()

# Create a heatmap to visualize the correlations
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", linewidths=0.
5)
plt.title("Feature Correlation Heatmap")

# Define a correlation threshold (e.g., 0.7) above which features are cons
idered highly correlated
correlation_threshold = 0.7

# Find highly correlated feature pairs
highly_correlated_features = set()
for i in range(len(correlation_matrix.columns)):
    for j in range(i):
        if abs(correlation_matrix.iloc[i, j]) > correlation_threshold:
            colname = correlation_matrix.columns[i]
```

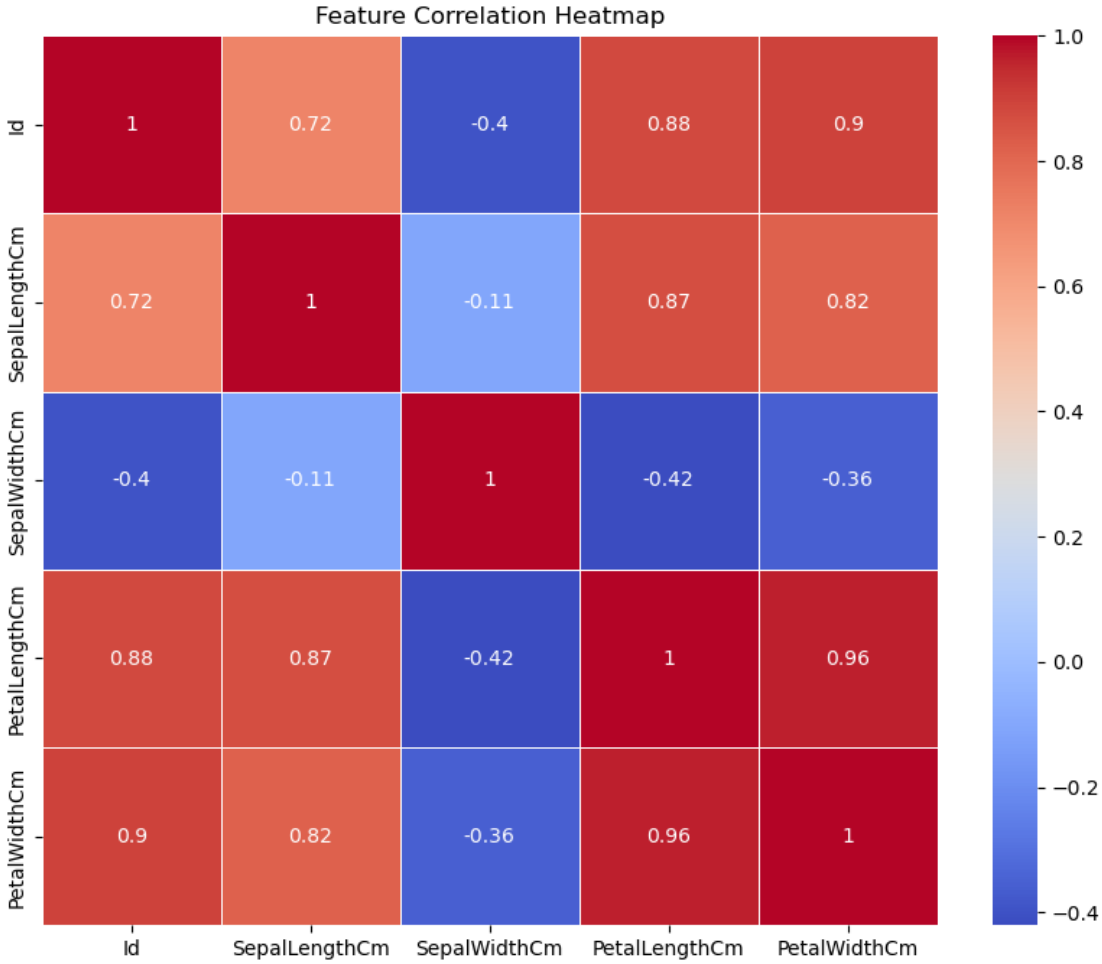
```
            highly_correlated_features.add(colname)

# Print the highly correlated feature pairs
print("Highly Correlated Features:")
print(highly_correlated_features)

# Drop one feature from each correlated pair
X_filtered = X.drop(columns=highly_correlated_features)

# Now, X_filtered contains the dataset with highly correlated features rem
oved
```

```
Highly Correlated Features:
{'PetalWidthCm', 'SepalLengthCm', 'PetalLengthCm'}
```



4.2: Feature Importance(wrapper Approach)

You can get the feature importance of each feature of your dataset by using the feature importance property of the model. Feature importance gives you a score for each feature of your data, the higher the score more important or relevant is the feature towards your output variable. Feature importance is an inbuilt class that comes with Tree Based Classifiers, we will be using Extra Tree Classifier for extracting the top 10 features for the dataset.

```
import numpy as np
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.datasets import load_iris
```

```python
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load the Iris dataset
data = load_iris()
X = data.data  # Features
y = data.target  # Target variable

# Split the data into a training and testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, r
andom_state=42)

# Create an ExtraTreesClassifier
clf = ExtraTreesClassifier(n_estimators=100, random_state=42)

# Fit the classifier to the training data
clf.fit(X_train, y_train)

# Calculate feature importances
feature_importances = clf.feature_importances_

# Sort features by importance in descending order
sorted_indices = np.argsort(feature_importances)[::-1]

# Select the top 'k' most important features (e.g., top 2)
k = 2
selected_feature_indices = sorted_indices[:k]

# Select the corresponding features from the original data
X_train_selected = X_train[:, selected_feature_indices]
X_test_selected = X_test[:, selected_feature_indices]

# Train and evaluate a classifier using the selected features
clf_selected = ExtraTreesClassifier(n_estimators=100, random_state=42)
clf_selected.fit(X_train_selected, y_train)
y_pred = clf_selected.predict(X_test_selected)

# Calculate accuracy with the selected features
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy with {k} selected features: {accuracy:.2f}")
```

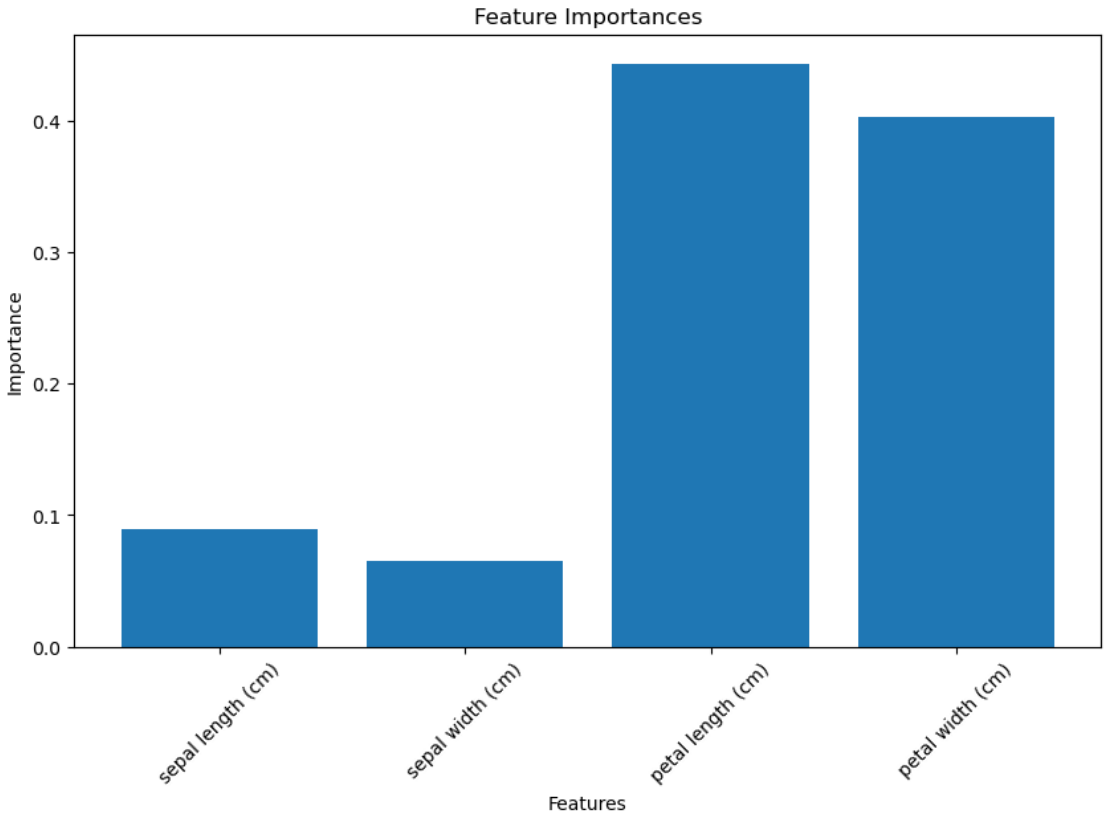
```
Accuracy with 2 selected features: 1.00
```

```python
# Calculate feature importances
feature_importances = clf.feature_importances_

# Get feature names
feature_names = data.feature_names

# Create a bar graph to visualize feature importances
plt.figure(figsize=(10, 6))
plt.bar(range(len(feature_importances)), feature_importances, tick_label=f
eature_names)
plt.xlabel("Features")
```

```
plt.ylabel("Importance")
plt.title("Feature Importances")
plt.xticks(rotation=45)
plt.show()
```



4.3: Univariate Selection(Filter approach)

Univariate feature selection is a filter-based approach for feature subset selection. It assesses the statistical relationship between each feature and the target variable independently, without considering interactions between features. One common technique for univariate feature selection is using the SelectKBest class from scikit-learn. It selects the top k features with the highest scores based on a specified statistical test.

```
# Import necessary libraries
import numpy as np
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2

# Load a sample dataset (Iris dataset in this example)
data = load_iris()
X = data.data  # Features
y = data.target  # Target variable
feature_names = data.feature_names

# Create SelectKBest object and specify the number of features to select (
e.g., 2)
k = 2
selector = SelectKBest(score_func=chi2, k=k)

# Fit the selector to the data and transform the features
```

```
X_new = selector.fit_transform(X, y)

# Get the indices of the selected features
selected_feature_indices = selector.get_support(indices=True)

# Get the column names of the selected features
selected_feature_names = [feature_names[i] for i in selected_feature_indic
es]

# Print the column names of the selected features
print("Column names of selected features:", selected_feature_names)
```

```
Column names of selected features: ['petal length (cm)', 'petal width (cm)
']
```